

目录

一、python 处理图像的实用工具或框架.....	2
(1) Scipy.....	2
(2) Pgmagick.....	2
(3) Numpy.....	2
(4) Opencv-python.....	2
(5) Simplecv.....	2
二、图像处理的主流方法.....	3
三、利用 python 的使用工具或框架处理照片.....	3
(1) Opencv-python.....	3
(2) Remove.bg.....	5
四、简谈利用 Python 的实用工具或框架进行图像处理的心得体会.....	7

一、python 处理图像的实用工具或框架

关于 python 与图像处理，通过网上查找，以下将列举五个基于 python 的处理图像的实用工具或框架，以便于加深关于 python 与图像处理之间的联系，有利于进一步学习通过实用 python 的实用工具去处理相关图像的基本方法。

（1）Scipy

Scipy 是一个用于数学、科学、工程领域的常用软件包，可以处理插值、积分、优化、图像处理、常微分方程数值解的求解、信号处理等问题。Scipy 可以有效的计算 Numpy 矩阵，故它和 Numpy 可以协同工作，因此可以凭借两者的协同工作，处理相关图片。

因为 Scipy 与 Numpy 协同工作的原因，Scipy 的开发效率非常高。另一方面，对于初学图像处理的人，Scipy 较为易懂，方便学习。但 Scipy 是提供的是一个作科学计算的工具集，它必须搭配一基础库，运用时限制大

（2）Pgmagick

Pgmagick 是 GraphicsMagick 库基于 Python 的包装器。Pgmagick 提供了强大而高效的工具和库集合，支持超过 88 种主要格式图像的读取、写入和操作，包括 DPX, GIF, JPEG , , PNG, PDF 等重要格式。在图像处理方面，Pgmagick 也可以做到图片的缩放与图片的边缘提取。

正如上所说，Pgmagick 支持超过 88 种主要格式的图像处理，这也时其相较于别种工具或者框架的优势所在。但同时也对 Pgmagick 的操作需求划定的较高的下限，对于初学者来说也较难上手。

（3）Numpy

Numpy 提供了许多向量和矩阵操作，能让用户轻松完成最优化、线性代数、积分、插值、特殊函数、傅里叶变换、信号处理和图像处理、常微分方程求解以及其他科学与工程中常用的计算，不仅方便易用而且效率更高。但是，Numpy 对其运行的对象具有一定的需求。

Numpy 突出的优点就是可以高效的直接的完成数组和矩阵的运算，且无需循环。而在通过浅显的学习于图像处理的相关知识，我们可以了解到，以计算机的视角来看待的图像是由大量的像素点集合而成，而像素点的亮度和其颜色通道可以视作矩阵。故 Numpy 对矩阵和数组的运算，也间接为图片处理提供了有力的帮助。

（4）Opencv-python

Opencv-python 可以实现计算机图像、视频的编辑。它也广泛应用于图像识别、运动跟踪、机器视觉等领域等。Opencv 是计算机视觉中实用最广泛的库之一，因为其后台由 c/c++ 编写的代码组成，故运行较快，且由于前端是 python 包装器，所以在编写代码时对于有一定 python 基础的较为方便且易学。

但 Opencv 的安装较为复杂，其中涉及的 opencv 库的下载与安装对应版本对应路径，还有其系统的环境变量的调试等等，对初学者较不友好。但其对图像的处理确实较为高效，且学习难度较低。

（5）Simplecv

SimpleCV 与 Opencv 相似，也是用于构建计算机视觉应用程序的开源框架。通过它可以访问如 OpenCV 等高性能的计算机视觉库，而无需首先了解位深度、文件格式或色彩空间等。学习难度远远小于 OpenCV。其优点是摄像机、视频文件、图像和视频流都可以交互操作，

语法简洁，可读性强，所以即使是初学者也可以编写简单的机器视觉测试。由于 SimpleCV 可以近似看作是 Opencv 的 python 精简版本，它简化了许多常见的任务，其缺点是但在编写较为复杂的大型程序，Simple 较不适用。

二、图像处理的主流方法

图像分割是计算机视觉研究中的一个经典难题,已经成为图像理解领域关注的一个热点,图像分割是图像分析的第一步,是计算机视觉的基础,是图像理解的重要组成部分,同时也是图像处理中最困难的问题之一。所谓图像分割是指根据灰度、彩色、空间纹理、几何形状等特征把图像划分成若干个互不相交的区域,使得这些特征在同一区域内表现出一致性或相似性,而在不同区域间表现出明显的不同。简单的说就是在一副图像中,把目标从背景中分离出来。对于灰度图像来说,区域内部的像素一般具有灰度相似性,而在区域的边界上一般具有灰度不连续性。

Python 中 PIL 提供了通用的图像处理功能,以及大量有用的基本图像操作,可以对图像进行缩放、裁剪、旋转、颜色转换等。

而在图像上绘制点、直线和曲线时, Matplotlib 是个很好的类库,具有比 PIL 更强大的绘图功能。Matplotlib 可以绘制出高质量的图表,就像本书中的许多插图一样。。

对于图像处理的实质, NumPy 是其计算工具包,用来表示向量、矩阵、图像等以及线性代数函数,这即是图像处理的实质,即是对数组矩阵的处理。NumPy 中的数组对象实现数组中重要的操作,比如矩阵乘积、转置、解方程系统、向量乘积和归一化,这为图像变形、对变化进行建模、图像分类、图像聚类提供了基础。

PCA 是有用的降维技巧。它可以在使用尽可能少维数的前提下,尽量多地保持训练数据的信息,在此意义上是一个最佳技巧。PCA 产生的投影矩阵可以被视为将原始坐标变换到现有的坐标系,坐标系中的各个坐标按照重要性递减排列。

SciPy 是建立在 NumPy 基础上,用于数值运算的开源工具包。SciPy 提供很多高效的操作,可以实现数值积分、优化、统计、信号处理,以及对我们来说最重要的图像处理功能。图像模糊。

以上 PIL, Matplotlib, NumPy, PCA, SciPy 是在 python 中处理图像较为主流的方法。而关于如何利用 python 的使用工具或框架进行图像前景及背景的有效分离,在查阅过相关的资料后,了解到现在主流方法是运用 opencv 中的 grabcut 算法和利用 requests 库请求目标网站 Remove.bg(智能工具)进行前景背景分离。

三、利用 python 的使用工具或框架处理照片

(1) Opencv

运用 Opencv 处理图像需要在 python 中安装 opencv-python 模块。并用 opencv 中先进行前景框定,后 grabcut 算法对图像进行前景背景的分离。在使用 grabcut 算法时应当了解 cv2.grabCut() 中的各个参数,如图所示:

```
cv2.grabCut(img, mask, rect_copy, bgdModel, fgdModel, 5, cv2.GC_INIT_WITH_RECT)
```

img()指的是带处理的图像,括号中输入该图片的路径,使 python 读取。

Mask 指的是掩码图像,在运行时,将设定的前景和背景保存到 mask 中,之后再传入 grabCut 函数进行处理。故对于分离前景和背景的判断,主要的靠的是掩码图像。所以在掩码图像中只能取 4 个值,即是 0 (背景), 1 (前景), 2 (可能的背景), 3 (可能的前景)

Rect 指的是所框定的区域视作 ROI,只有被框定的部分才被处理,以外的区域系统默认为背景,即为 0。

bgdmodel 与 fgdmodel 分别指的是背景模型与前景模型,两者必须是单通道浮点型图像。图中的 5 即是 iterCount,其为迭代次数,迭代次数要合适,才使前景和背景分离得当。

GC_INIT_WITH_RECT 为用矩形窗初始化 GrabCut,即为显示。

了解的 grabCut 中各个参数的意义,以方便获取数据。

由于在不同图片中所需框定的 ROI 区域并不一致，所以 `rect` 中的四个参数也不一致，为在图里不同的图片中不因为 ROI 区域的不一致而不断改变参数，不断调试。在程序中加鼠标回调函数，可以进行交互鼠标选取 ROI。完整代码如下所示

```
import cv2
import numpy as np

def on_mouse(event, x, y, flag, param):
    global rect
    global leftButtonDown
    global leftButtonUp

    if event == cv2.EVENT_LBUTTONDOWN:
        rect[0] = x
        rect[2] = x
        rect[1] = y
        rect[3] = y
        leftButtonDown = True
        leftButtonUp = False

    if event == cv2.EVENT_MOUSEMOVE:
        if leftButtonDown and not leftButtonUp:
            rect[2] = x
            rect[3] = y

    if event == cv2.EVENT_LBUTTONUP:
        if leftButtonDown and not leftButtonUp:
            x_min = min(rect[0], rect[2])
            y_min = min(rect[1], rect[3])

            x_max = max(rect[0], rect[2])
            y_max = max(rect[1], rect[3])

            rect[0] = x_min
            rect[1] = y_min
            rect[2] = x_max
            rect[3] = y_max
            leftButtonDown = False
            leftButtonUp = True

img = cv2.imread('F:\whjmsn4\opencvpicture\cztt6.png')

mask = np.zeros(img.shape[:2], np.uint8)

bgdModel = np.zeros((1, 65), np.float64)

fgdModel = np.zeros((1, 65), np.float64)

rect = [0, 0, 0, 0]

leftButtonDown = False
leftButtonUp = True

cv2.namedWindow('img')
cv2.setMouseCallback('img', on_mouse)
cv2.imshow('img', img)

while cv2.waitKey(2) == -1:
    if leftButtonDown and not leftButtonUp:
        img_copy = img.copy()
        cv2.rectangle(img_copy, (rect[0], rect[1]), (rect[2], rect[3]), (0, 255, 0), 2)
        cv2.imshow('img', img_copy)
    elif not leftButtonDown and leftButtonUp and rect[2] - rect[0] != 0 and rect[3] - rect[1] != 0:
        rect[2] = rect[2] - rect[0]
        rect[3] = rect[3] - rect[1]
```

```

rect_copy = tuple(rect)
rect = [0, 0, 0, 0]
cv2.grabCut(img, mask, rect_copy, bgdModel, fgdModel, 5, cv2.GC_INIT_WITH_RECT)

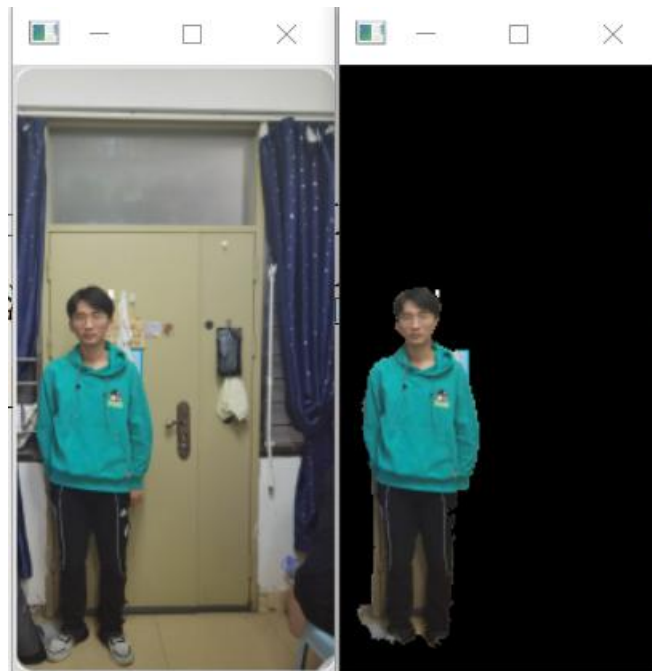
mask2 = np.where((mask == 2) | (mask == 0), 0, 1).astype('uint8')
img_show = img * mask2[:, :, np.newaxis]

cv2.imshow('grabcut', img_show)
cv2.imshow('img', img)
cv2.imshow('mask', mask)
cv2.imshow('mask2', mask2)

cv2.waitKey()
cv2.destroyAllWindows()

```

该程序便可以实现图像前景背景分离。当程序中 `mask` 及 `mask2`，掩模图像 `mask` 元素值已经变成了 0~3 之间的值。值为 0 和 2 的将转为 0，值为 1 和 3 的将转为 1，然后保存在 `mask2` 中，这样就可以用 `mask2` 过滤出所有的 0 值像素(理论上会保存所有的前景像素)。即为掩模图像，运行代码，可以得到如下所示几张图：



上图的在边缘部分的灰度极具相似性，所以前景和背景的分离效果还是较为不好的，但也基本完成了前景和背景的分离。

(2) Remove.bg

先需要知道何为 `remove.bg` 是什么？通过网络查询我们可以简单的了解到 `Remove.bg` 基于 Python、Ruby 和深度学习技术开发而来。它通过强大的 AI 人工智能算法实现自动识别出前景主体与背景图，并将它们分离开来。`Remove.bg` 是较为成熟的前景与背景的分离技术，我们可以通过 python 的 `requests` 库来运用 `Remove.bg`。`request` 是 python 实现的简单易用的 HTTP 库，实用起来较为简洁，由于是第三方库，与 `opencv-python` 一样，需要通过 cmd 安装。`Request.get()`用于请求目标网站，类型是 `HTTPResponse` 类型。

通过运用 `Remove` 与 `request`，我们可以编写出代码，如图所示：

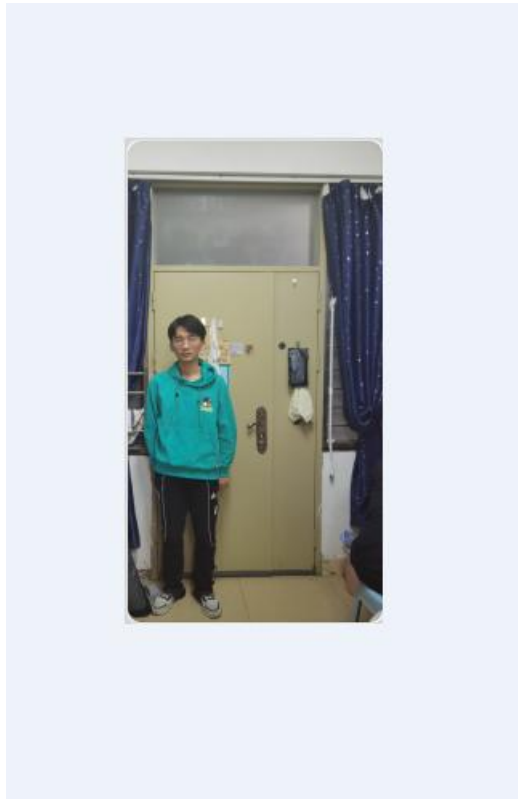
```

import requests

response = requests.post(
    'https://api.remove.bg/v1.0/removebg',
    files={'image_file': open('F:\whjmsn4\opencvpicture\czt5.png', 'rb')},
    data={'size': 'auto'},
    headers={'X-Api-Key': 'k3b7XvwFSiteVNZgxPpRkzvL'},
)
if response.status_code == requests.codes.ok:
    with open('F:\whjmsn4\opencvpicture\czt5.png', 'wb') as out:
        out.write(response.content)
else:
    print("Error:", response.status_code, response.text)

```

在运行代码后，会直接对原图进行修改，原图如下所示：



在运行代码后，前景背景进行分离，结果如下图所示：



对比 `opencv-python`，基于深度学习的 `Remove.bg` 在前景和背景分离运用上较为完全，分离效果更好。

三、简谈利用 Python 的实用工具或框架进行图像处理的心得体会

关于 `python` 的实用工具或框架的对图像处理的运用，通过本次学习，不仅对 `python` 的运行原理有了一定程度的认识，也对图像处理的延展方面，如视频处理，抑或是图像的识别也建立了一定的兴趣。相关本此学习的心得体会，会选取遇到困难较多的几部分进行分析，包括下载与安装，理解代码，运行排错，总结梳理这四个方面。从自己的心得体验中再度获取经验。

（1）下载与安装

一开始的目标是通过上网查询资料，然后是给自己指定为学习 `opencv-python`，通过 `python` 对图像进行前景背景分离需要在 `python` 中安装 `opencv-python` 模块。而证明是否安装成功的结果是在 `python` 的 `idle` 中输入 `import cv2`，运行后不报错，则证明模块安装成功。安装 `opencv-python` 模块需要从运行中输入 `cmd` 后再输入 `pip install opencv-python`。这边我出现的第一次错误，就在在输入 `pip` 指令是，系统路径并不是 `python` 文件中有 `pip3` 的路径，故无论怎么检查，得出的一直是没有 `pip` 这指令的报错。

在解决路径的问题后，我成功的将 `opencv-python` 在 `cmd` 中安装成功。紧接的是是第二次的错误，`dll` 模块不存在，上网查询资料后说明是需要 `vs2015` 环境，但放在下载可能需要的是 `vs2020` 环境，于是便下载的 `vs`，并按网上资料所说调试了环境变量。但是 `dll` 模块依旧不存在。

至此，便改换个思路，再从安装出发，找不同的安装方法。之后选择的是 `Anaconda` 的安装，按照网上说法，`Anaconda` 是一个包括 `python` 的工具的大集合，且其与之前不同的是，它在安装的时候会将环境配置好，所以必须初学者将环境一个一个的安装，可以直接在 `Anaconda Prompt` 中找到相关的路径，直接安装 `opencv-python`。这次便直接安装成功了。

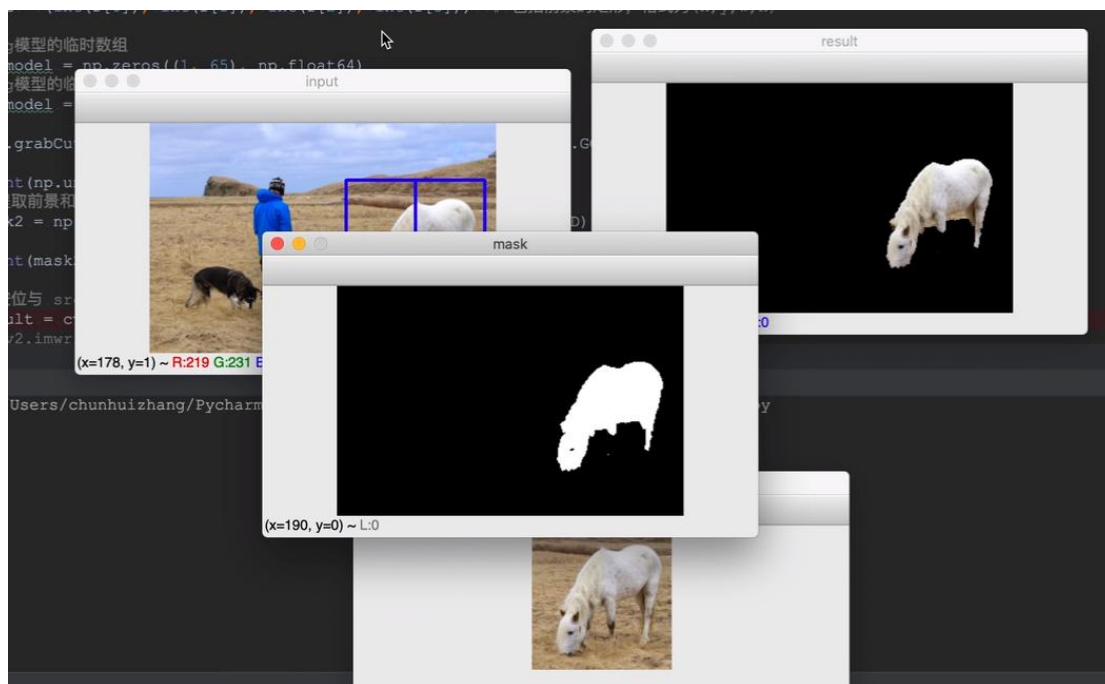
通过这几次安装失败的过程，也并不一无所获，从中知道在 `cmd` 中输入相关的路径的必要性，也知道 `python` 的运行并不是仅仅是代码的输入，代码的运行，若要达成某种目的，必要模块的安装，环境变量的调试，也是 `python` 能够正常运行的必要条件。调试也扩展知识面，可以通过 `Anaconda` 中的 `Prompt` 和 `jupyter notebook` 来运行 `python`。

（2）理解代码

虽然本此学习还是基于 `python`，但是在代码的理解上还是造成一定的麻烦。其中较为显著的便是 `grabcut` 算法的理解。正如上文所说，理解 `grabcut` 算法，要了解其中各个参数的意思，但还是有个大前提是要理解计算机时如何看图像。在学习的过程中，便把这两者是顺序给看反了，因此在理解代码这一方面花费了较多的时间。在关于计算机如何看待图像，在互联网上浅显的学习，知道计算机读取图像时，是将其看成众多的像素点，而像素点因其的颜色通道，还有亮度或者称为灰度，亮度和灰度的程度可以用 0 到 255 表示。因此图片也可以看作一个一个矩阵组成。

了解了计算机是如何读取图像的，再反过来看 `grabcut` 中的各个参数，便明了了不少。例如，`rect` 所截取的区域为何可以称为 ROI，还有为 `rect` 中四个参数表示的是是什么，通过学习，可以知道 `rect` 前两的参数表示的是所截取区域的起点，后两个数表示所截取区域的长和宽。但也有不懂的，便须要再去查找资料，关于 `grabcut` 知识的相关讲解。

在 `grabcut` 中，在本次学习中较难以理解的便是 `mask` 在图像中是如何在前景和背景分离中起作用的。在代码中可以看出 `mask` 在分离中有极其重要的地位，但是无论是理论学习抑或是视频讲解，终归是不太了解 `mask` 的作用，掩码图像在图像前景和背景分离的作用。因此在资料查询中特别留意有无实际操作演示的。所查到的演示如下图所示：



在 `python` 以及其它程序代码的学习中，学习理论知识，理解代码固然重要，但还是要通过实际的操作与实践，才可真正的了自己所编写的代码到底是如何运行，这在理解代码中显得十分重要。

（3）运行排错

这次的运行由于用的是 `Anaconda`，换了一个性的运行环境。这次代码的运行是通过 `jupyter notebook` 进行的。开始时弹出个类似与 `cmd` 小黑框，其中会有一网址，将其输入浏览器，接下来的操着便和 `python` 中 `idle` 相差不大。但这次学习如何运行的却较为新鲜。

在 `opencv-python` 的排错较为简易，前提是了解其中代码的意思，改变合适的参数截取合适的 ROI 区域。而关于 `Remove.bg` 的排错便放了一个较大错误